

Lab 4 — Rapport individuel — Gustave Perdigeon — DEVOPS-3

Nom : Gustave Perdigeon

Formation : ESILV — A4 OCC

Date : 2 octobre 2026

Sujet individuel : DEVOPS-3 — Endpoint /boom et gestion globale des erreurs

Introduction

Ce laboratoire avait pour objectif de mettre en pratique un travail en équipe avec Jira, GitHub et une application Node.js utilisant Express. Jira permettait de suivre les tâches, tandis que GitHub servait à partager les modifications et à les faire vérifier avant leur intégration. Des tests automatisés et une chaîne d'intégration continue permettaient de contrôler le code.

Ma tâche était DEVOPS-3, consacrée à la route /boom et à la gestion globale des erreurs. Cette route déclenche volontairement une erreur afin de vérifier que l'application renvoie une réponse cohérente. La fonctionnalité étant déjà fournie dans le projet, mon travail a principalement porté sur sa vérification et sur le renforcement de son test d'intégration.

1. Plan — Organisation du travail dans Jira

Notre équipe était composée de quatre membres. Le travail était réparti en tickets Jira, avec un tableau permettant de suivre les étapes To Do, In Progress, Review et Done. Mon ticket DEVOPS-3 concernait la vérification du statut HTTP 500 et du contenu JSON renvoyé lorsqu'une erreur est déclenchée par /boom.

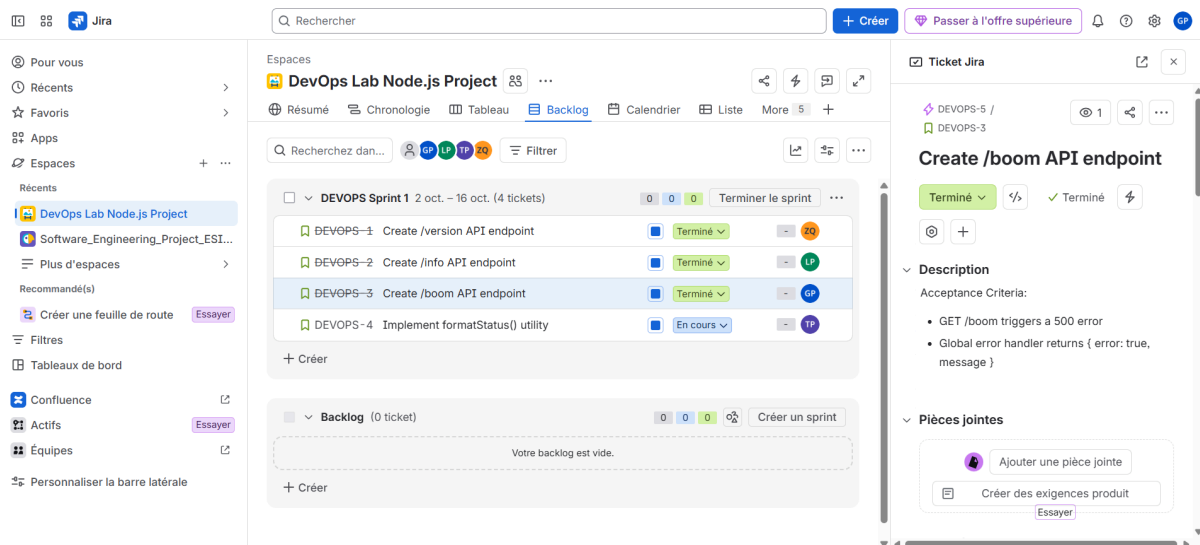


Figure 1 — Ticket Jira DEVOPS-3 et suivi de la tâche.

2. Code — Travail sur une branche et vérification de la fonctionnalité

J'ai récupéré le dépôt du projet et installé ses dépendances avec npm. J'ai ensuite créé la branche `feature/DEVOPS-3-boom-endpoint` afin de travailler sur ma tâche sans modifier directement la branche principale. L'identifiant DEVOPS-3 dans le nom de la branche et dans les messages de commit permet de relier les modifications à la tâche Jira.

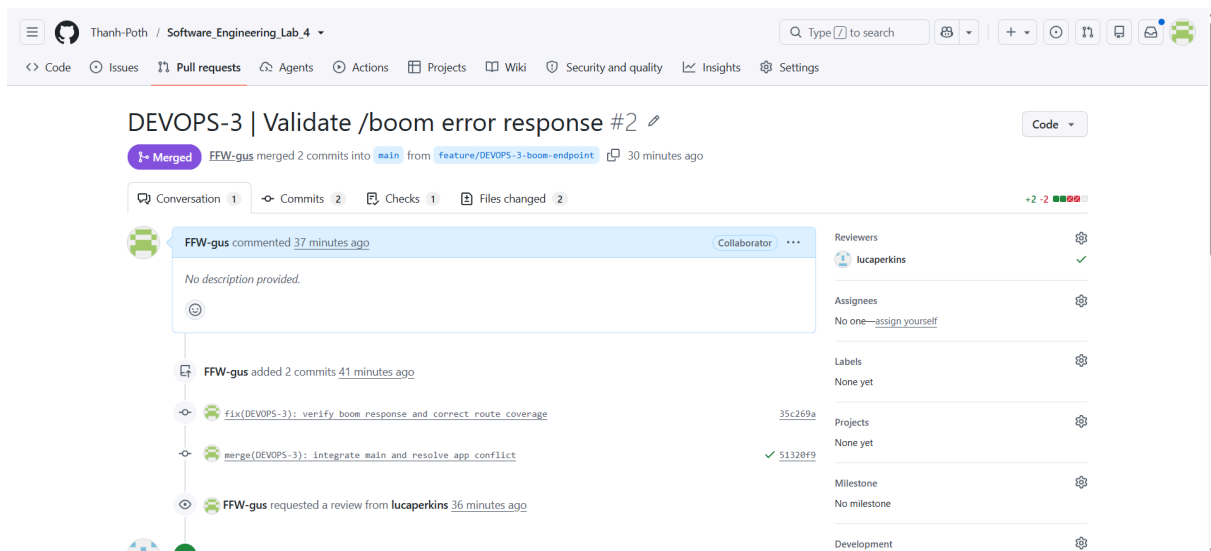


Figure 2 — Branche de travail et commits associés à DEVOPS-3.

Fonctionnement du code fourni

La route est définie dans `src/routes/auto/boom.route.js`. Lorsqu'une requête `GET /boom` est reçue, elle crée une erreur avec le message « Boom! », lui attribue le statut 500 puis la transmet à Express avec `next(err)`. Le gestionnaire situé dans `src/utls/errorHandler.js` récupère cette erreur et renvoie un objet JSON contenant `error: true` et le message. Ce gestionnaire est enregistré après les routes dans `src/app.js`.

```
1  /**
2   * GET /boom → triggers an error to exercise the global error handler.
3   */
4  import { Router } from 'express'
5
6  const router = Router()
7
8  router.get('/boom', (req, _res, next) => {
9    const err = new Error('Boom!')
10    err.status = 500
11    next(err)
12  })
13
14  export default router
15
```

```
1  /**
2   * Centralized Express error handler.
3   * Ensures consistent JSON error shape across the API.
4   */
5  export function errorHandler (err, _req, res, _next) {
6    const status = err?.status ?? 500
7    const message = err?.message ?? 'Internal Server Error'
8    res.status(status).json({ error: true, message })
9  }
10
```

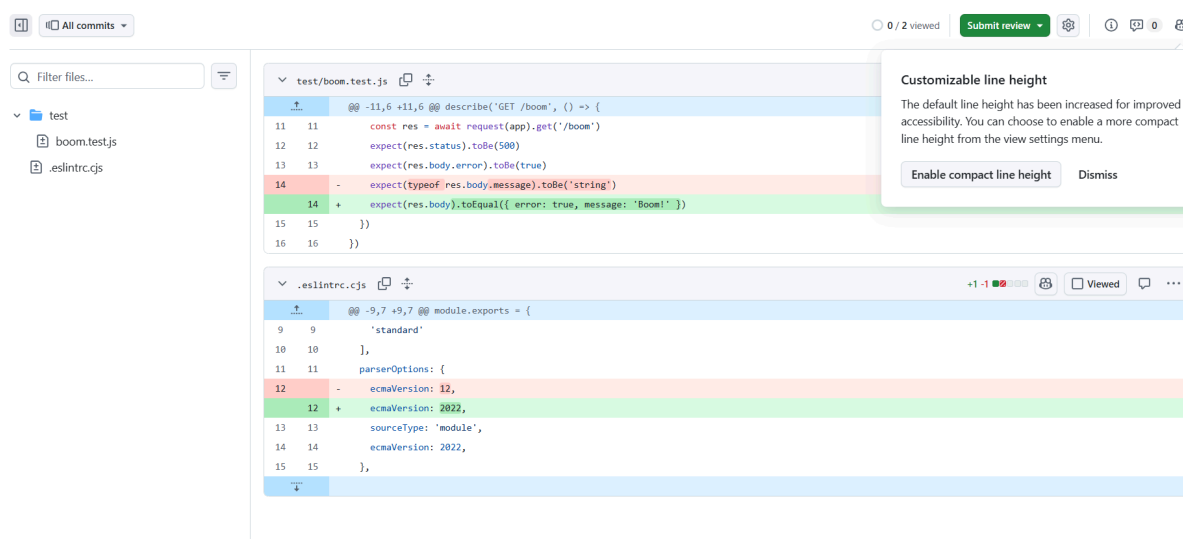
Figure 3 — Route `/boom` et gestionnaire global d'erreurs fournis dans le projet.

Modification personnelle du test

Le test d'intégration existant vérifiait le statut 500, la présence d'une erreur et le fait que le message était une chaîne de caractères. J'ai renforcé cette

dernière vérification afin de contrôler exactement l'objet JSON attendu. Le test détecte ainsi une réponse dont le message serait incorrect ou dont la structure aurait changé.

```
expect(res.body).toEqual({ error: true, message: 'Boom!' })
```



Preuve du lab 4

Figure 4 — Renforcement du test d'intégration de /boom.

J'ai également adapté la configuration ESLint en passant `ecmaVersion` de 12 à 2022 pour permettre l'analyse de la syntaxe JavaScript utilisée par le chargement des routes.

Vérification locale

Le comportement attendu lors d'un appel à `/boom` est une réponse HTTP 500 accompagnée du JSON suivant :

```
{"error": true, "message": "Boom!"}
```

J'ai également exécuté l'application localement et vérifié que `GET /boom` renvoyait bien le statut HTTP 500 et le message attendu.

```
PS C:\Users\gusta\Documents\Software engineering\lab4\Software_Engineering_Lab_4> npm.cmd run dev

> devops-lab-student-project@1.0.0 dev
> nodemon src/index.js

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node src/index.js`
[server] listening on http://localhost:3000
```

Preuve du lab 4

```
Problems  Output  Debug Console  Terminal  Ports

PS C:\Users\gusta\Documents\Software engineering\lab4\Software_Engineering_Lab_4> curl.exe -i http://localhost:3000/boom
HTTP/1.1 500 Internal Server Error
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 32
ETag: W/"20-8yyCetSx3ISyvQTUz0o3PYzFNts"
Date: Fri, 02 Oct 2026 11:07:33 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"error":true,"message":"Boom!"}
PS C:\Users\gusta\Documents\Software engineering\lab4\Software_Engineering_Lab_4>
```

Preuve du lab 4

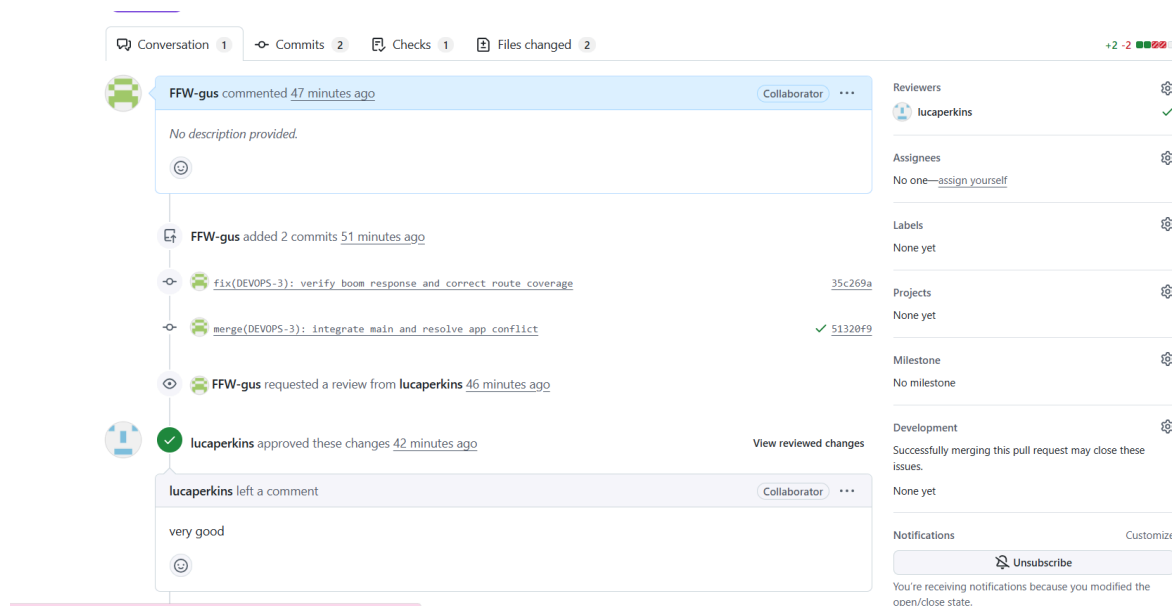
Figure 5 — Réponse HTTP de /boom lors de la vérification locale.

3. Build — Intégration des modifications avec GitHub

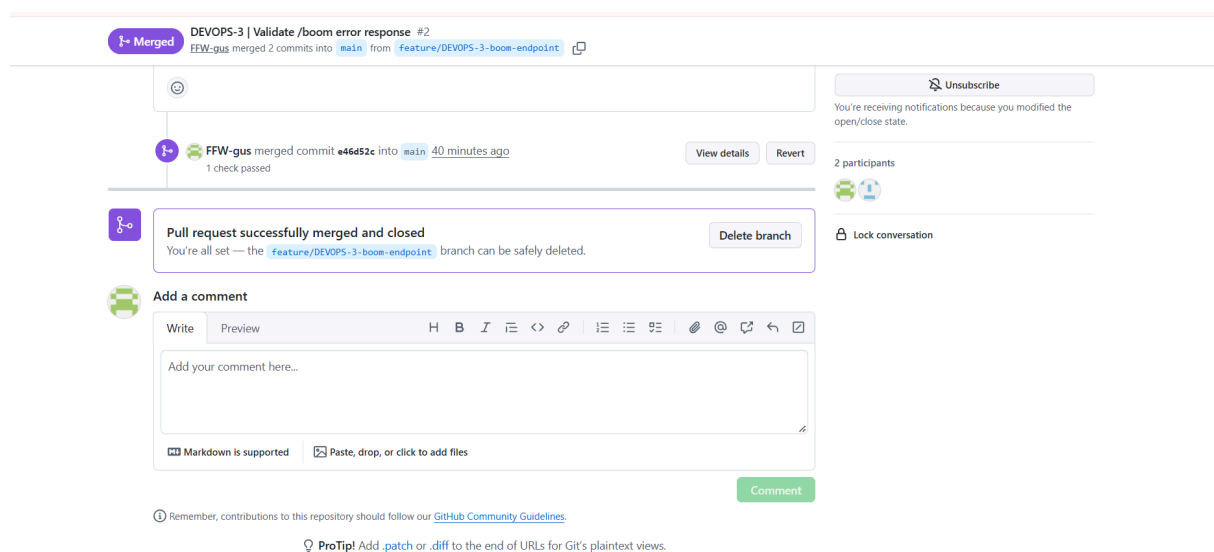
J'ai enregistré mes modifications dans Git et envoyé ma branche sur GitHub. Avant de terminer l'intégration, j'ai récupéré les modifications de main. Un conflit est apparu dans `src/app.js` car le chargement des routes avait été modifié de deux côtés. J'ai conservé la version provenant de main, qui charge les routes avec un import dynamique et `pathToFileURL`, tout en conservant ma modification du test. J'ai ensuite relancé le lint et les tests avant de finaliser la fusion.

J'ai ouvert la pull request n° 2, intitulée « DEVOPS-3 | Validate /boom error response ». Un collègue a approuvé la demande et la vérification automatique a

réussi. La pull request a ensuite été fusionnée dans main. J'ai enfin mis à jour ma branche principale locale avec git pull --ff-only origin main.



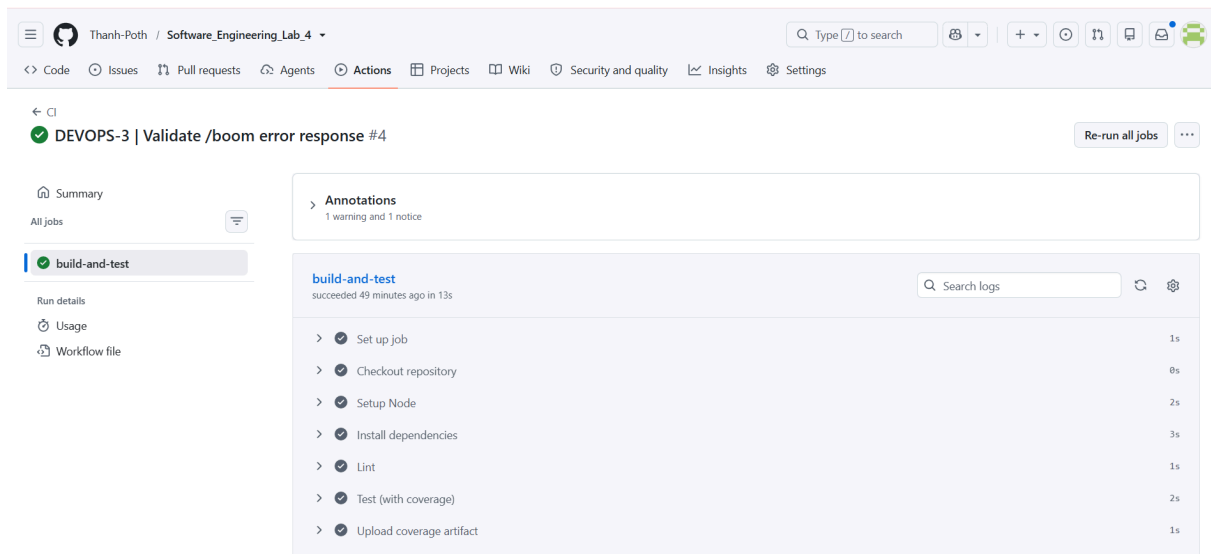
Preuve du lab 4



Preuve du lab 4

Figure 6 — Approbation, vérification automatique et fusion de la pull request.

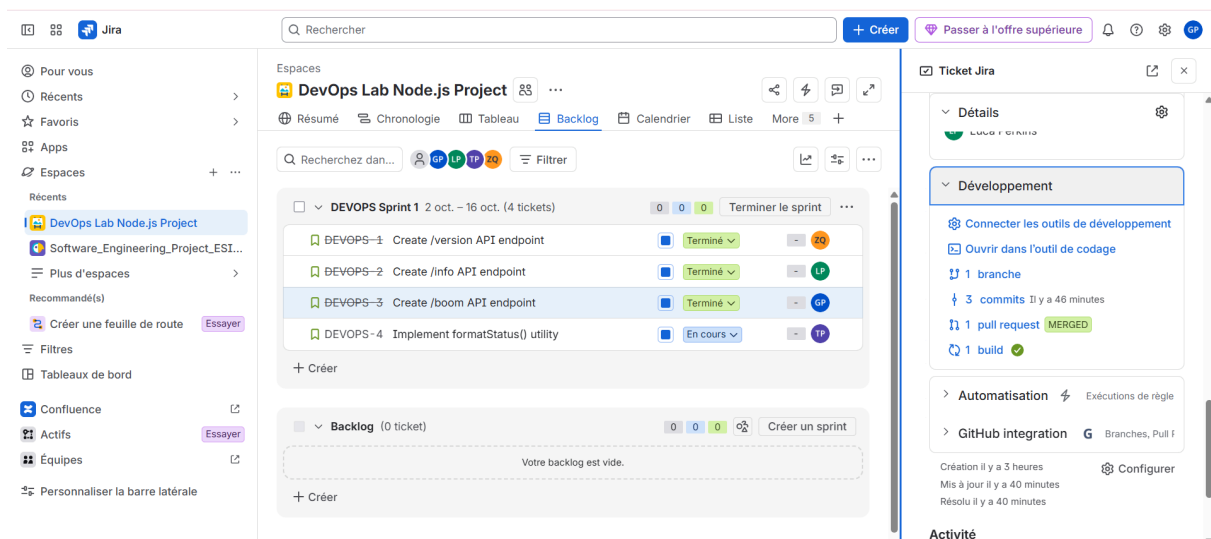
Le workflow d'intégration continue défini dans .github/workflows/ci.yml automatise l'installation des dépendances, le contrôle du style avec ESLint et l'exécution des tests avec leur couverture. Il permet de vérifier les modifications dans un environnement commun à toute l'équipe.



Preuve du lab 4

Figure 7 — Exécution réussie du workflow d'intégration continue.

La rubrique Développement du ticket DEVOPS-3 affiche une branche, trois commits, une pull request fusionnée et un build réussi. Cette liaison permet de retrouver le travail GitHub depuis Jira.



Preuve du lab 4

Figure 9 — Éléments GitHub associés au ticket DEVOPS-3.

4. Test — Résultats et couverture

Les tests utilisent Vitest. Les tests unitaires du gestionnaire d'erreurs vérifient son comportement lorsqu'aucun statut ni message n'est fourni, puis lorsqu'un statut et un message spécifiques sont présents. Le test d'intégration utilise

Supertest pour envoyer une requête à l'application Express et contrôler la réponse complète de /boom.

Après la résolution du conflit, ESLint n'a signalé aucune erreur et les 15 tests répartis dans 7 fichiers ont réussi. La couverture globale obtenue était de 93,18 % pour les lignes, 92,15 % pour les instructions, 81,81 % pour les fonctions et 72,22 % pour les branches. Ces résultats dépassaient les seuils de 80 % pour les lignes, instructions et fonctions, et de 70 % pour les branches. La route /boom et le gestionnaire d'erreurs étaient couverts à 100 % dans ce rapport.

```
PS C:\Users\gusta\Documents\Software engineering\lab4\Software_Engineering_Lab_4> npm.cmd run dev

> devops-lab-student-project@1.0.0 dev
> nodemon src/index.js

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node src/index.js`
[server] listening on http://localhost:3000
Terminer le programme de commandes (O/N) ? o
PS C:\Users\gusta\Documents\Software engineering\lab4\Software_Engineering_Lab_4> npm.cmd run lint

> devops-lab-student-project@1.0.0 lint
> eslint .

PS C:\Users\gusta\Documents\Software engineering\lab4\Software_Engineering_Lab_4> npm.cmd test -- --coverage

> devops-lab-student-project@1.0.0 test
> vitest run --coverage

RUN v4.1.11 C:/Users/gusta/Documents/Software engineering/lab4/Software_Engineering_Lab_4
Coverage enabled with v8

✓ test/unit/appInfo.test.js (3 tests) 11ms
✓ test/unit/errorHandler.test.js (2 tests) 11ms
✓ test/unit/status.test.js (4 tests) 10ms
✓ test/boom.test.js (1 test) 32ms
✓ test/info.test.js (2 tests) 53ms
✓ test/health.test.js (1 test) 36ms
✓ test/version.test.js (2 tests) 70ms

Test Files 7 passed (7)
Tests 15 passed (15)
```

Preuve du lab 4

```
✓ test/health.test.js (1 test) 50ms
✓ test/version.test.js (2 tests) 70ms

Test Files 7 passed (7)
Tests 15 passed (15)
Start at 13:12:54
Duration 992ms (transform 417ms, setup 0ms, import 2.10s, tests 223ms, environment 1ms)

% Coverage report from v8
-----
File | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
-----
All files | 92.15 | 72.22 | 81.81 | 93.18 |
src | 80.95 | 37.5 | 50 | 82.35 |
  app.js | 94.44 | 50 | 66.66 | 100 | 25-31
  index.js | 0 | 0 | 0 | 0 | 8-10
src/routes/auto | 100 | 100 | 100 | 100 |
  boom.route.js | 100 | 100 | 100 | 100 |
  info.route.js | 100 | 100 | 100 | 100 |
  version.route.js | 100 | 100 | 100 | 100 |
src/utills | 100 | 100 | 100 | 100 |
  appInfo.js | 100 | 100 | 100 | 100 |
  errorHandler.js | 100 | 100 | 100 | 100 |
  status.js | 100 | 100 | 100 | 100 |
-----
PS C:\Users\gusta\Documents\Software engineering\lab4\Software_Engineering_Lab_4> |
```

Preuve du lab 4

Figure 8 — Résultats des tests et couverture du code.

Bilan individuel

Ce laboratoire m'a permis de pratiquer un cycle de contribution avec Git et GitHub : créer une branche liée à un ticket, vérifier le comportement du code, renforcer un test, envoyer mes modifications puis les faire relire avant leur fusion. La résolution du conflit m'a également montré l'importance de récupérer les modifications de l'équipe et de relancer les contrôles après l'intégration. Ma contribution principale a été la vérification de la gestion des erreurs et l'amélioration de la précision du test de /boom.

Références

[Dépôt GitHub du projet](#)

[Pull request individuelle n° 2](#)